Προαιρετικό Project HY-280: Συμπίεση DFA για Deep Packet Inspection

Χαράλαμπος Ε. Τσουρακάκης

HY-280 · Θεωρία Υπολογισμού

Υπεύθυνος ΤΑ: Noah Wahl
Βασισμένο στο paper του Kumar et al., SIGCOMM 2006 [1].

Προαιρετικό Project - Παράδοση 18/12 για Bonus 5%

Bonus

Το project είναι **προαιρετικό** και προσφέρει έως **+5%** στον τελικό βαθμό.

Σε ποιους απευθύνεται;

Σε φοιτητές που απολαμβάνουν την έρευνα και τις απαιτητικές προγραμματιστικές προκλήσεις, και θέλουν να υλοποιήσουν έναν αλγόριθμο από ερευνητική εργασία.

Η σύστασή μου

Αν δυσκολεύεστε με την ύλη, επικεντρωθείτε πρώτα στο **100% του βαθμού από ασκήσεις και εξετάσεις**. Το project είναι για όσους θέλουν να πάνε ένα βήμα παραπέρα.

Γιατί μας ενδιαφέρει το περιεχόμενο της δικτυακής κίνησης;



- Το IP μεταφέρει πακέτα από μία μηχανή σε μία άλλη [2].
- Το TCP δίνει στην εφαρμογή μία αξιόπιστη, διατεταγμένη ροή από bytes [3].
- Για ασφάλεια, δεν αρκεί πάντα να ξέρουμε *πού* πηγαίνει η κίνηση. Θέλουμε να κοιτάξουμε και *τι περιέχει*.

IP και TCP: από πακέτα σε ροή bytes

Παράδειγμα

Μία εφαρμογή στέλνει `HELLO WORLD`. Το TCP μπορεί να το μεταφέρει σε διαφορετικά πακέτα, π.χ.

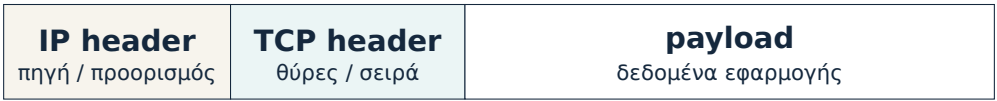
`HELLO` `WORLD`,

αλλά ο παραλήπτης βλέπει ξανά τη διατεταγμένη ροή

`HELLO WORLD`.

IP: πακέτα $\Longrightarrow$ TCP: διατεταγμένη ροή bytes

Header και payload



Deep Packet Inspection

Για να εντοπίσουμε υπογραφές επιθέσεων, μπορούμε να ψάχνουμε μέσα στο **payload** για πρότυπα που περιγράφονται με κανονικές εκφράσεις.

Γιατί $|\Sigma| = 256$;

Ένα byte = 8 bits

Κάθε byte έχει $2^8 = 256$ δυνατές τιμές:

$$\boxed{\Sigma = \{0, 1, 2, \ldots, 255\}}, \qquad |\Sigma| = 256.$$

Byte value	Hex	Όταν ερμηνεύεται ως ASCII
10	0A	newline
32	20	space
65	41	A
71	47	G
97	61	a
...	...	...

Το DFA δεν βλέπει «γράμματα». Βλέπει μία ακολουθία από byte values.

Το DFA επεξεργάζεται ένα byte κάθε φορά

Για κάθε τρέχουσα κατάσταση q και επόμενο byte x:

$$q' = \delta(q, x).$$



Το πλεονέκτημα

Ένα DFA χρειάζεται ουσιαστικά ένα transition lookup ανά byte.

Cisco security: Snort 3

Το Snort είναι open-source intrusion detection / prevention engine της Cisco Talos και ο ίδιος inspection engine χρησιμοποιείται στο Cisco Secure Firewall.



- Cisco: *Use Regular Expressions in BGP*; Cisco Secure Firewall Snort 3 intrusion-policy documentation.
- Snort 3 Rule Writing Guide: `content`, `pcre`, `regex`, `fast_pattern`.

Anatomy μιας Snort rule

Ένα απλοποιημένο defensive example:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS (
msg:"Repeated path traversal candidate";
http_uri;
regex:"/(?:\.\.\/){2,}/i",fast_pattern;
sid:1000001;
)
```



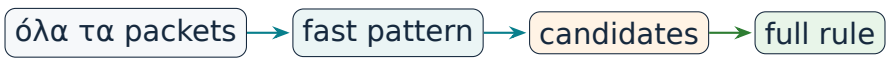
Προσοχή

Ένα signature είναι ένδειξη για συγκεκριμένο observable pattern. Δεν είναι απόδειξη για την πρόθεση του sender ούτε πλήρης ανίχνευση κάθε παραλλαγής επίθεσης.

Snort 3: regex μέσω Hyperscan

Η Snort 3 documentation αναφέρει ότι το rule option `regex` κάνει matching σε payload/application data μέσω της **Hyperscan search engine**.

- Η syntax είναι PCRE-like για το υποστηριζόμενο subset.
- Ένα regex μπορεί να επισημανθεί ως `fast pattern`.
- Η engine μπορεί να χρησιμοποιήσει το pattern για γρήγορο αρχικό filtering πριν ακριβότερο rule evaluation.



Γιατί τα αυτόματα είναι φυσικά για intrusion detection;

Ένα high-throughput IDS πρέπει να διαβάζει stream bytes και να ελέγχει **πολλά patterns ταυτόχρονα**.

$$\text{byte}_1, \text{byte}_2, \ldots \quad \longrightarrow \quad \text{state updates}$$

Το ιδανικό

Μικρή, προβλέψιμη εργασία ανά byte και καθόλου ανάγκη να αποθηκεύουμε ολόκληρο το stream.



Συγκρατούμε τα εξής:

Αυτό είναι ακριβώς το computation model ενός finite automaton.

Το πρόβλημα: πολλά regex μπορούν να κάνουν το DFA τεράστιο

Αν συνδυάσουμε πολλά patterns και κάνουμε determinization, ο αριθμός states μπορεί να αυξηθεί δραματικά.



Για DPI, η μνήμη είναι μέρος της ταχύτητας

Αν το transition table δεν χωρά σε cache / fast memory, η θεωρητικά $O(1)$ μετάβαση μπορεί να γίνει πρακτικά ακριβή.

Το πρόβλημα: το transition table είναι μεγάλος

Για κάθε state πρέπει να γνωρίζουμε τι γίνεται για κάθε μία από τις 256 τιμές byte:

$$\delta(q, 0), \delta(q, 1), \ldots, \delta(q, 255).$$

Άρα ένα πλήρες DFA αποθηκεύει

$$\boxed{256|Q| \text{ transition entries.}}$$

Αν κάθε destination state id απαιτεί 4 bytes:

$$\boxed{\text{memory} \approx 256 \cdot |Q| \cdot 4 \text{ bytes.}}$$

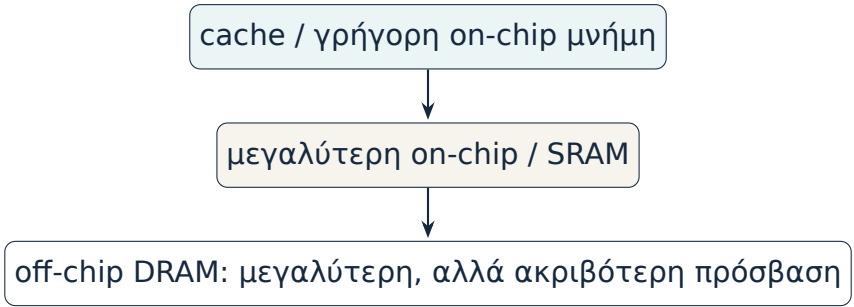
Πόσο γρήγορα μεγαλώνει η μνήμη;

$\lvert Q \rvert$	Transition entries	4 bytes / entry
$1{,}000$	$256{,}000$	≈ 0.98 MiB
$10{,}000$	2.56 M	≈ 9.8 MiB
$100{,}000$	25.6 M	≈ 97.7 MiB
$1{,}000{,}000$	256 M	≈ 0.95 GiB

Άρα

Το bottleneck μπορεί να γίνει η **μνήμη**: χωρητικότητα, cache locality και memory bandwidth --- όχι η λογική του DFA.

Πού αποθηκεύεται ο transition table;



Όσο μεγαλώνει ο πίνακας, τόσο δυσκολότερο είναι να παραμένει κοντά στον επεξεργαστή. Σε high-throughput matching, κάθε επιπλέον memory access έχει σημασία.

Βασική ιδέα των Kumar et al.

Πολλές γραμμές του transition table είναι σχεδόν ίδιες.

State u

x	a	b	c	d
$\delta(u,x)$	q_4	q_5	q_2	q_7

State v

x	a	b	c	d
$\delta(v,x)$	q_4	q_5	q_6	q_7

Οι δύο γραμμές συμφωνούν στα a, b, d. Γιατί να αποθηκεύσουμε ξανά τις ίδιες μεταβάσεις;

Η ιδέα της συμπίεσης

Για το state v κρατάμε μόνο τη διαφορά

$$v \xrightarrow{c} q_6$$

και προσθέτουμε ένα **default link**

$$v \dashrightarrow u.$$

Αν έρθει το byte a:

$$v \dashrightarrow u \xrightarrow{a} q_4.$$

Κρίσιμο

Το default link **δεν καταναλώνει** το input byte. Το ίδιο byte a χρησιμοποιείται όταν φτάσουμε στο u.

Το παράδειγμα του paper

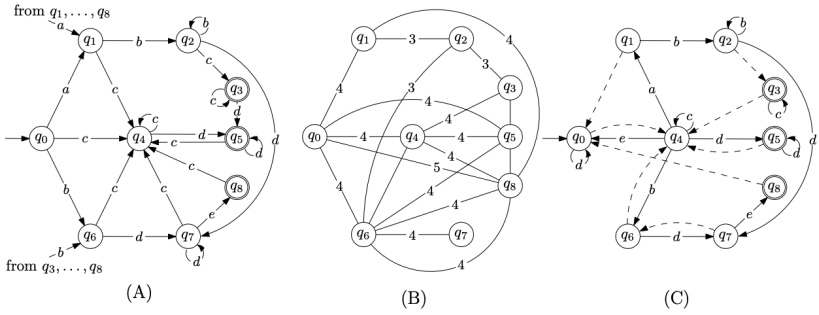


Figure 1: Example from [29]. (A) DFA D for regular expression `.*((ab+c+)|(cd+)|(bd+e))`. Edges to q_0 are omitted. (B) Space reduction graph for D with edges annotated with similarity. Edges with similarity less than 4 omitted, except those connecting q_2 to avoid disconnecting the graph. (C) D^{2}FA equivalent to D. All transitions are shown, default transitions are dashed.

(A) αρχικό DFA, (B) graph ομοιότητας, (C) ισοδύναμο D²FA με dashed default links. Το σχήμα αναπαράγεται στους Bille--Gørtz--Pedersen και προέρχεται από το παράδειγμα των Kumar et al. [4, 1]

Η συνολική ροή του project



Τι σας δίνεται;

Ο generator, τα συγκεκριμένα seeds και τα official DFA instances παρέχονται από εμάς.

Implement → **test** → **measure** → **prove** → **report**

Τι σημαίνει επιτυχημένο project;

Να κατανοήσετε το paper,
Να κατασκευάσετε έναν σωστό compressor,
να τον ελέγξετε πειραματικά,
να αποδείξετε ότι δεν αλλάζει τη γλώσσα,
και **να εξηγήσετε** το space--lookup trade-off.

$$\text{Algorithms} + \text{Systems} + \text{Formal Verification}$$

Γιατί αυτό το project είναι ενδιαφέρον;

- **Θεωρία αυτομάτων στην πράξη:** τα DFA παύουν να είναι μόνο διαγράμματα και γίνονται πραγματικά εργαλεία για pattern matching.
- **Ανάγνωση ερευνητικής εργασίας:** καλείστε να κατανοήσετε και να υλοποιήσετε έναν αλγόριθμο από paper.
- **Αλγοριθμική σκέψη:** πρέπει να καταλάβετε πώς η συμπίεση εκμεταλλεύεται ομοιότητες μεταξύ transition rows.
- **Systems intuition:** βλέπετε πώς η μνήμη, το locality και το κόστος lookup επηρεάζουν την απόδοση ενός πραγματικού συστήματος.

Το βασικό μήνυμα

Δεν υλοποιούμε απλώς έναν αλγόριθμο: καταλαβαίνουμε *γιατί* χρειάζεται και *πότε* βοηθά.

Από το testing στη formal verification

- **Testing και reproducibility:** γράφετε `pytest`, δουλεύετε με fixed seeds και παράγετε επαναλήψιμα experiments.
- **Formal verification:** μαθαίνετε τη διαφορά ανάμεσα στο

  ``το δοκίμασα σε πολλές εισόδους''

  και στο

  ``το απέδειξα για όλες τις εισόδους''.
- **Tradeoff analysis:** δεν αρκεί να συμπιέσετε το DFA, πρέπει να εξηγήσετε τι κερδίζετε σε μνήμη και τι πληρώνετε σε επιπλέον lookup work.

Ερευνητική νοοτροπία

Το project απαιτεί όχι μόνο σωστό κώδικα, αλλά και αιτιολόγηση, πειράματα και απόδειξη ορθότητας.

Ένα 6-week mini research project

Theory $\rightarrow$ Algorithm $\rightarrow$ Implementation $\rightarrow$ Experiments $\rightarrow$ Proof

- Κατανοείτε το θεωρητικό αντικείμενο.
- Υλοποιείτε έναν αλγόριθμο από τη βιβλιογραφία.
- Ελέγχετε πειραματικά την υλοποίησή σας.
- Μετράτε πραγματικά tradeoffs.
- Πιστοποιείτε στη Lean ότι η συμπιεσμένη αναπαράσταση είναι ισοδύναμη με την αρχική.

Στο τέλος

Έχετε **υλοποιήσει, μετρήσει, ελέγξει και πιστοποιήσει** ένα πραγματικό αλγοριθμικό σύστημα.

Υλικό του project

github.com/tsourolampis/
cs280-theory-of-computation/tree/main/project

Στο repository θα βρείτε

- τον generator των DFA instances, το verification script για τα instances,
- το paper των Kumar et al.,
- το starter αρχείο για τη Lean,
- το `README.md` με τις οδηγίες και τα παραδοτέα.

Ξεκινήστε από εδώ

Διαβάστε πρώτα το `README.md` και ακολουθήστε τη ροή του project βήμα προς βήμα.

Σημαντικό concept: γιατί τα tests δεν αρκούν;

Μπορείτε να ελέγξετε:

- 1.000 strings,
- 1.000.000 strings,
- όλα τα strings μέχρι κάποιο μήκος.

Αλλά η γλώσσα έχει άπειρα πιθανά strings Σ^*

Άρα

Τα tests δίνουν πολύ καλή **εμπιστοσύνη και debugging**, αλλά δεν αποτελούν μαθηματική απόδειξη ισοδυναμίας για όλες τις εισόδους.

Πιστοποιήστε την ισοδυναμία στη Lean

Για κάθε original DFA M και compressed automaton C, θέλουμε να αποδείξετε ότι συμπεριφέρονται το ίδιο. Η απόδειξη χωρίζεται σε δύο ιδέες:

1. **Local correctness:**

$$\forall q, x, \qquad \mathrm{lookup}_C(q, x) = \delta_M(q, x).$$

2. **Whole-word correctness:** με επαγωγή στο input word,

$$\forall q, w, \qquad \mathrm{run}_C(q, w) = M.\mathrm{evalFrom}(q, w).$$

Από έναν πεπερασμένο έλεγχο σε όλες τις δυνατές εισόδους

Για κάθε state q και byte x ελέγχουμε ότι

$$\text{lookup}_C(q, x) = \delta_M(q, x).$$

Αυτό απαιτεί μόνο

$$|Q| \cdot |\Sigma| = 256|Q|$$

τοπικούς ελέγχους.

Η κρίσιμη ιδέα

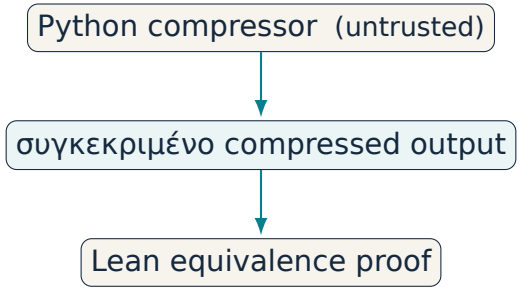
Αφού αποδείξουμε την ορθότητα για **ένα symbol**, μία επαγωγή στη λέξη δίνει ορθότητα για **κάθε** $w \in \Sigma^*$.

$$\underbrace{\forall q, x}_{\text{finite local check}} + \underbrace{\text{induction on } w}_{\text{generic proof}} \Longrightarrow \underbrace{\forall q, w}_{\text{all inputs}}$$

Τι ακριβώς πιστοποιεί η Lean;

Η Lean μας παρέχει το εργαλείο για να **αποδείξουμε** πως το output του Python compressor είναι σωστό. Ουσιαστικά γράφουμε μια επαγωγική απόδειξη ως Lean πρόγραμμα!

- Μην χάσετε το 2ο φροντιστήριο αυτή την Παρασκευή πάνω σε Lean.

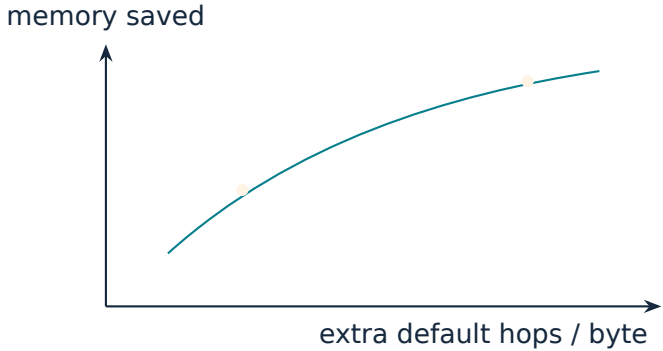


Τι πρέπει να περιέχει η αναφορά;

Τεχνική αναφορά, οδηγίες στο Git (template+σελίδες):

1. σύντομη περιγραφή του προβλήματος
2. σύντομη περιγραφή του αλγορίθμου που υλοποιήσατε,
3. design choices,
4. testing strategy,
5. ιδέα της Lean απόδειξης,
6. experimental setup,
7. plots / tables,
8. ερμηνεία του memory--lookup trade-off,
9. limitations / τι θα αλλάζατε.
10. βιβλιογραφικές αναφορές και AI disclaimer

Σημείωση : Το βασικό trade-off που πρέπει να δείτε πειραματικά



λιγότερη μνήμη	$\Longleftrightarrow$	πιθανώς περισσότερα lookups

Δεν αρκεί να πείτε «το compressed είναι μικρότερο». Θέλουμε να εξηγήσετε **τι κερδίζουμε και τι πληρώνουμε**.

Παραδοτέα

1. Python source: loader, baseline DFA, Kumar compressor
2. `pytest` test suite.
3. Compressed JSON outputs για τα official instances.
4. Lean files που πιστοποιούν την ισοδυναμία κάθε required pair input DFA και compressed output
5. Reproducible experiment scripts και CSV/plots.
6. `README.md` με ακριβείς εντολές εκτέλεσης.
7. Τεχνική αναφορά
8. Σύντομο live demo / oral defense.

Βιβλιογραφία

[1] S. Kumar, S. Dharmapurikar, F. Yu, P. Crowley και J. Turner. ``Algorithms to Accelerate Multiple Regular Expressions Matching for Deep Packet Inspection''. Στο: *Proceedings of ACM SIGCOMM*. 2006, σσ. 339–350.

[2] J. Postel. *Internet Protocol*. Αδημοσίευτη ερευνητική εργασία RFC 791. RFC Editor, 1981.

[3] W. Eddy. *Transmission Control Protocol (TCP)*. Αδημοσίευτη ερευνητική εργασία RFC 9293. RFC Editor, 2022.

[4] P. Bille, I. L. Gørtz και M. R. Pedersen. *Fast Practical Compression of Deterministic Finite Automata*. Figure 1 reproduces the D2FA example of Kumar et al. 2025.